

SOP — Red Flag Diagnostics

Audience: Mike (Data Manager) and Aaron (Owner).

Purpose: When something looks wrong with the daily run, this doc tells you what to check first, what each symptom means, and who owns the fix.

The escalation path:

1. Mike spots a red flag during morning triage
 2. Mike checks this doc, runs the diagnostic command(s)
 3. If diagnostic confirms the issue, ping Aaron in Slack with the section number
 4. Aaron fixes (typically <30 min), confirms in Slack, run resumes
-

How to read this doc

Each red flag below has 4 parts:

- **Symptom** — what Mike sees
 - **What it means** — root cause
 - **Diagnostic** — exact command(s) to run from `/Users/aaron/Desktop/SiftStack`
 - **Fix owner** — who handles it (Mike vs. Aaron)
-

1. No Slack post by 8:45 AM

Symptom: No "🇺🇸 SiftStack Ohio Daily" post in Slack/Discord by 8:45 AM.

What it means: One of three things —

- (a) The Apify Actor didn't fire (cron schedule issue, Apify outage, billing)
- (b) The Actor fired but crashed before reaching the Slack notify step
- (c) The Actor finished but the Slack webhook URL is broken

Diagnostic:

1. Open [Apify Console](#). Find the SiftStack Actor → "Runs" tab. Check if today's 6:00 AM run exists.
 - **No run at all** → cron schedule misfired or Actor disabled. Manually trigger via "Start" button.
 - **Run exists, status = SUCCEEDED** → Slack webhook broken. Check `SLACK_WEBHOOK_URL` in Apify Actor input.
 - **Run exists, status = FAILED** → check the run logs. Top-level error usually identifies the source (e.g., one scraper threw, DataSift login failed, etc.).
2. If Apify is down, run the pipeline manually from your laptop:

```
bash cd /Users/aaron/Desktop/SiftStack source venv/bin/activate python src/main.py daily --upload-datasift --notify-slack
```

Fix owner: Aaron (Apify cron, webhook). Mike can manually trigger as a stopgap.

2. Record count is unusually low

Symptom: Slack reports e.g. 8 records when the typical day is 40–60.

What it means:

- (a) **Specific portal is down or changed format** — one or more scrapers returned 0 records
- (b) **Genuine quiet day** (Mondays after holiday weekends, weather events, etc.) — happens 2–4×/year
- (c) **Date filter mis-set** (`--since` accidentally set to today instead of last 7 days)

Diagnostic:

1. Look at the **county breakdown** in the Slack post. Is one county zero?

- If Franklin = 0: probably Franklin auditor or Franklin scraper. Test:

```
bash PYTHONPATH=src python -m scrapers.oh_franklin_probate --days 7 PYTHONPATH=src python -m scrapers.oh_franklin_foreclosure --days 7
```

- If Montgomery = 0: probably go.mcoho.org down. Test:

```
bash PYTHONPATH=src python -m scrapers.oh_montgomery_probate --days 7 PYTHONPATH=src python -m scrapers.oh_montgomery_foreclosure --days 7
```

- If Greene = 0 on probate but >0 on foreclosure: **expected** — Greene probate is gated at the source (license disabled). The sentinel scraper logs a warning when it flips back on.

2. Compare against the **last 7 days of CSVs** in `output/` — is today an outlier?

```
bash ls -la /Users/aaron/Desktop/SiftStack/output/*.csv | head -10 wc -l
/Users/aaron/Desktop/SiftStack/output/*.csv | head -10
```

3. If a specific portal is broken, check the portal manually in a browser. If the HTML structure changed, the scraper needs an update.

Fix owner:

- Cause (a): Aaron (scraper code update — typically 1–2 hours)
- Cause (b): Nobody — note in Slack and move on
- Cause (c): Aaron (CLI args / cron config)

3. Enrichment failures (missing Zillow / Smarty / phone data)

Symptom: Records in DataSift have empty Zestimate / property type / phone fields where you'd expect them.

What it means:

- (a) **API key out of credits / expired** — Tracerfy, Trestle, OpenWebNinja all have monthly quotas
- (b) **API rate limit hit** — temporary, usually self-resolves next run
- (c) **Specific record has unusable address** — Smarty couldn't standardize, downstream APIs refuse to query

Diagnostic:

1. Check the Apify run log for warning lines:

- `TRACERFY OUT OF CREDITS` → Tracerfy out

- `OPENWEBNINJA_API_KEY` missing → key not set or invalid
- `Trestle scoring failed` → Trestle API issue
- `Smarty standardization skipped` → Smarty creds issue

2. Check API account balances:

- Tracerfy: <https://tracerfy.com/billing>
- Trestle: <https://trestleiq.com> (dashboard)
- OpenWebNinja: <https://openwebninja.com> (dashboard)
- Smarty: <https://smarty.com> (dashboard)

3. For specific record holes: open the record in DataSift, check the Notes field — the pipeline writes which enrichment steps succeeded/failed per record.

Fix owner:

- (a) Aaron (top up credits, ~5 min per service)
- (b) Nobody (next run resolves)
- (c) Mike (for individual records — manual lookup if high-priority)

4. Tag drift (records not matching expected niche sequential preset)

Symptom: Apply preset "01. SMS Day 1", expecting 19 records, see 0 or wildly different count.

What it means:

- (a) **Tags didn't apply during upload** — DataSift column mapping issue (Tags column wasn't mapped during the upload wizard)
- (b) **Preset filter logic changed** — someone edited the preset and it now filters differently
- (c) **Records are in DataSift but in the wrong list** — list column wasn't mapped during upload

Diagnostic:

1. Open one of today's records in DataSift. Check the **Tags** field at the top of the record:

- Should see: `Courthouse Data, probate, montgomery, 2026-04, deceased, high_confidence, ...` (or similar — see [SOP-TAG-FLOW.md](#) for full tag inventory)

- **No tags at all** → Tags column wasn't mapped during upload. Re-run the upload manually:

```
bash python src/main.py csv-import --csv-path output/today_csv.csv --upload-datasift
```

Then watch the upload wizard — at "Map Columns" step, manually map the Tags column.

2. Run the preset discovery script to verify presets look correct:

```
bash python src/main.py manage-presets --discover
```

Should show all 21 presets (12 niche + 9 bulk) with their filter criteria.

3. If a specific preset is broken, fix it directly in DataSift UI:

- **00 Niche Sequential Marketing** folder → click broken preset → Edit Filters → verify "Property Status" excludes "Sold"

Fix owner:

- (a) Aaron (re-upload with manual column mapping — 5 min)
 - (b) Aaron (rebuild preset)
 - (c) Aaron (re-upload)
-

5. Probate records have no property address

Symptom: Probate records in DataSift show empty Property Street / City / ZIP fields.

What it means:

- (a) **OH Auditor lookup didn't run** — module import error or Apify environment doesn't have the auditor module
- (b) **OH Auditor lookup ran but couldn't find a match** — decedent didn't own property in their own name (common when family transferred title before death)
- (c) **OH Auditor portal is down or rate-limiting**

Diagnostic:

1. Check the Apify run log for these lines:

- `— Step 3c: Probate Property Lookup (N candidates) —`
- `Property address found: X/N`
- If "Step 3c" line is missing → module didn't run. Check that `src/probate_property_lookup.py` and `src/enrichment/oh*_auditor.py` are in the Docker image.
- If found = 0/N → all lookups missed. Check auditor portal manually.

2. Manually test the auditor for a specific decedent:

```
bash PYTHONPATH=src python -m enrichment.oh_montgomery_auditor "PATRICIA CRIDGE"
```

3. Note: for some probate records, the decedent genuinely didn't own real property in the county (executor lives elsewhere, property already in trust, etc.). **Up to 20% miss rate is normal.** For those, Mike asks the executor on the first call.

Fix owner:

- (a) Aaron (module deploy / Dockerfile fix)
- (b) Mike (manually ask executor on first call — this is a feature, not a bug)
- (c) Aaron if portal is consistently down; otherwise auto-resolves next run

6. DataSift upload didn't complete

Symptom: Slack post shows record count but DataSift dashboard has no new records.

What it means:

- (a) **DataSift login failed** during upload (password rotated, account locked)
- (b) **Upload wizard hung** (DataSift UI lag, modal blocking)
- (c) **CSV had structural issues** that DataSift rejected (invalid headers, encoding)

Diagnostic:

1. Apify run log → search for "DataSift" — last successful step indicates how far we got:

- "DataSift login OK" + nothing after = wizard hung
- "Upload File step OK" + "Map Columns failed" = column mapping issue
- No DataSift lines = login failed at the very start

2. Manually re-run upload only:

```
bash # The CSV is still in output/ - just push it PYTHONPATH=src python -c " from
```

```
datasift_uploader import upload_to_datasift upload_to_datasift('output/oh_daily_2026-04-27.csv', list_name='Probate', headless=False) "
```

Run with `headless=False` to watch what's happening in the browser.

3. If DataSift login fails, check `DATASIFT_EMAIL` / `DATASIFT_PASSWORD` in Apify Actor input.

Fix owner: Aaron. Mike can manually upload via the DataSift UI as a stopgap (drag the CSV into the Upload File panel).

7. Slack post says "0 records" but you know it should have hits

Symptom: Slack reports 0 records on a normal weekday.

What it means: Catastrophic — every scraper failed simultaneously. Almost always either:

- (a) **Apify environment broken** (network outage, missing dependency, expired secret)
- (b) **All portals temporarily down** (rare, but happens — usually Friday afternoon when state IT does maintenance)

Diagnostic:

1. Check Apify run log — should see exception traces for each of the 6 scrapers

2. Manually test one scraper from your laptop:

```
bash cd /Users/aaron/Desktop/SiftStack source venv/bin/activate PYTHONPATH=src python -m scrapers.oh_montgomery_probate --days 7
```

- If this works locally but Apify shows 0 → Apify environment issue

- If this also returns 0 → genuinely all portals are down (give it 30 min, retry)

3. If catastrophic, run the full pipeline locally to recover the day:

```
bash python src/main.py daily --upload-datasift --notify-slack
```

Fix owner: Aaron immediately. This is a P0.

8. Phone tier distribution looks wrong

Symptom: Slack reports "Tier 5 (Drop): 35" out of 47 records — way too many drops.

What it means:

- (a) **Trestle API returned junk** — temporary issue or bad batch
- (b) **All records in this batch genuinely have bad numbers** — possible if Tracerfy returned mostly disconnected lines (rare but happens with elderly decedents)
- (c) **Trestle scoring threshold misconfigured**

Diagnostic:

1. Open one Tier 5 record in DataSift. Look at the phone field — is it formatted correctly? If it shows e.g. "5551234567" without formatting, Trestle may have failed to score it.

2. Check Apify run log for `Trestle scored N unique phones across M DP records` — if N is much smaller than expected, Trestle had errors.

3. Check Trestle dashboard for API errors today.

Fix owner:

- (a) Aaron (re-run Trestle scoring on today's records)
 - (b) Mike (drop the batch, focus on Tier 1–3 only)
 - (c) Aaron (config fix — rare)
-

9. PDF deep-prospecting reports missing

Symptom: No new PDFs in `output/reports/` (or Google Drive if configured) for today's deceased records.

What it means:

- (a) **Report generator crashed** for one or more records (often layout/encoding)
- (b) **No deceased candidates today** — all records are foreclosures with living owners (PDFs only generate for deceased / heir / DM cases)
- (c) **Disk full** (output dir grows over time)

Diagnostic:

1. Apify run log → look for "PDF generation failed" warnings (per-record)
2. Check disk space: `df -h /Users/aaron/Desktop/SiftStack`
3. Re-generate PDFs from today's CSV:

```
```bash
PYTHONPATH=src python -c "
import csv
from models import NoticeData
from report_generator import generate_record_pdf
from pathlib import Path

notices = []
with open('output/oh_daily_2026-04-27.csv') as f:
 for row in csv.DictReader(f):
 n = NoticeData()
 for k,v in row.items():
 if hasattr(n,k): setattr(n,k,v)
 notices.append(n)

for n in notices:
 if n.owner_deceased == 'yes' or n.heir_map_json or n.decision_maker_name:
 try:
 generate_record_pdf(n, Path('output/reports'))
 except Exception as e:
 print(f'FAILED {n.address}: {e}')
"
```

**Fix owner:** Aaron. Low priority — Mike can work without PDFs for the day, just less efficient lead reading.

---

## 10. Greene probate scraper logs warning

---

**Symptom:** Apify log contains: `WARNING: Greene probate license re-enabled – re-test scraper before next run!`

**What it means:** The Greene County Probate Court re-enabled their public Case Search after we deployed the sentinel scraper. **Good news** — but the scraper needs to be updated from sentinel mode to real scraping mode.

**Diagnostic:** This is the OPPOSITE of an error — it means data is now available for Greene probate. Aaron needs to expand the scraper from health-probe to full implementation (~2–4 hour task).

**Fix owner:** Aaron. Not urgent (system continues to work — just missing one source until the upgrade ships).

---

## 11. RealAuction credentials expired or rotated

---

**Symptom:** Franklin foreclosure shows 0 records on a typical day. Apify log: `[Franklin] foreclosure – skipping: missing credentials (REALAUCTION_EMAIL, REALAUCTION_PASSWORD) or login error.`

**What it means:** Franklin foreclosure is the only OH source that needs an account login. RealAuction credentials may have expired, been rotated, or the account may need to re-verify.

**Diagnostic:**

1. Try logging in manually at <https://franklin.sheriffsaleauction.ohio.gov> with the credentials in `.env`
2. If login works, the issue is that the cookies expired in `realauction_cookies.json`. Delete it and re-run:  
`bash rm /Users/aaron/Desktop/SiftStack/realauction_cookies.json PYTHONPATH=src python -m scrapers.oh_franklin_foreclosure --days 7`
3. If login fails, you'll need to reset the password on RealAuction.

**Fix owner:** Aaron (RealAuction account management). Other 5 OH sources keep working in the meantime.

---

## 12. Tax Sale records have no preset / no automation

---

**Symptom:** Records sitting in "Tax Sale" list (40+/week from Montgomery) but no `FTM_TaxSale_*` preset exists, so the niche sequential cadence isn't picking them up.

**What it means:** Montgomery's foreclosure portal serves both mortgage foreclosures (sheriff sales) AND treasurer's tax sales on the same page. The scraper correctly classifies them as `tax_sale`, tags them `ftm-ts`, and routes them to the "Tax Sale" DataSift list. But Mike's preset structure was built around `FTM_LP_*` / `FTM_SS_*` / `FTM_Probate_*` only — there's no Tax Sale equivalent yet.

**Diagnostic:**

1. In DataSift, filter records by tag `ftm-ts` — confirm count matches the daily Slack summary's "tax\_sale" line

2. Open one record — verify it has the right tags ( `ftm` , `ftm-ts` , `montgomery` , etc.) and is in the "Tax Sale" list

**Fix:** Mike creates 3 new filter presets (~5 min each):

- `FTM_TaxSale_Mont` (filter: list=Tax Sale + tag `ftm-ts` + tag `montgomery` + County=Montgomery)
- `FTM_TaxSale_Franklin` (same but franklin/Franklin)
- `FTM_TaxSale_Greene` (same but greene/Greene — currently 0 records, but ready when Greene starts producing)

Or alternatively — if tax sales are operationally identical to sheriff sales for our acquisition workflow — Mike adds `tax_sale` records to the existing `FTM_SS_*` presets by updating the tag filter from `ftm-ss` to `ftm-ss` OR `ftm-ts` . **Tradeoff:** simpler operationally but loses analytical separation between tax-foreclosure and mortgage-foreclosure deal economics.

**Fix owner:** Mike (DataSift preset configuration).

---

## When you genuinely don't know what's wrong

---

The ONE-COMMAND diagnostic dump:



```
cd /Users/aaron/Desktop/SiftStack
source venv/bin/activate

1. Latest log
ls -lt logs/scrape_*.log | head -3
tail -100 $(ls -t logs/scrape_*.log | head -1)

2. Output CSV stats (last 7 days)
ls -la output/*.csv | head -10
for f in $(ls -t output/*.csv | head -3); do
 echo "=== $f ==="
 wc -l "$f"
 head -2 "$f"
done

3. Test each scraper individually (quick smoke)
for src in oh_montgomery_probate oh_montgomery_foreclosure oh_greene_foreclosure oh_franklin_probate
do
 echo "=== $src ==="
 PYTHONPATH=src python -c "
import asyncio
from scrapers.base import load_scraper
s = load_scraper('scrapers.$src')
print(f'OK: {s.county} {s.notice_type}')
```

Paste the output to Aaron in Slack with the suspected red-flag section number from this doc.

---

## Maintenance schedule (preventive)

---

To avoid most red flags, do these on schedule:

What	When	Owner
Top up Tracerfy credits (\$0.02/record × ~50/day = ~\$30/month)	Monthly	Aaron
Top up Trestle credits (\$0.015/phone × ~150/day = ~\$70/month)	Monthly	Aaron
Verify DataSift password not rotated	Quarterly	Aaron
Test all 6 scrapers locally (catch HTML-format changes early)	Monthly	Aaron
Review preset filter logic (drift catches)	Monthly	Mike
Clean <code>output/</code> directory of files older than 90 days	Quarterly	Aaron (or cron)

---

## See also

- [SOP-DAILY-OPERATIONS.md](#) — Mike's morning playbook
- [SOP-TAG-FLOW.md](#) — Exact tag/list/preset map
- [CLAUDE.md](#) — Full operational reference